Everything but the Software:

How Coding Agents Accelerate Feature-Driven Decay. An Exploratory Case Study and an Experiment Protocol

Juan Manuel Servera Bondroit*

Draft of 25 October 2026

Abstract

Can another agent provide the reflection and course correction that feature-driven delivery crowds out? I investigate this question through an exploratory case study and an experiment protocol. I propose that coding agents may accelerate software decay through compounding errors and output that exceeds human review capacity. Over six weeks I used agents, centred on Microsoft's open-source HyperVelocity Engineering (HVE Core) framework, to research a fictional housing retrofit programme, write requirements and deploy an MVP. I then handed 192 issues to a coordinator running parallel research–plan–implement–review loops. In ten days it merged 183 pull requests and the log grew by about 37,000 words, yet I could no longer use the system, while an automated rubric gave implementation completeness five out of five. I propose five mechanisms: specification inflation, proxy capture, learning evaporation, process-gate inflation and feedback starvation. The case motivates, but cannot answer, whether an additional agent can recognise drift and restore options for future change. I specify a bounded test: an agent reviews, refactors under test-commit-or-revert and rewrites a short design note after every feature. Feature-only, consolidation and review-only arms will be compared on correctness, internal quality, an unforeseen change request, human-intervention demand and cost. This tests agent-led consolidation, not autonomous product steering or the replacement of human judgement. The broader question is whether agents can help keep development on a useful course, rather than merely produce more work.

Keywords: agentic software engineering; coding agents; spec-driven development; software evolution; technical debt; refactoring; feedback loops; human–AI interaction; exploratory case study; experiment protocol.

1 Introduction

On 15 October 2026 I started an agent coordinator against a backlog of more than 180 GitHub issues and let it run. Each issue went into its own git worktree, where an agent session ran a full research, plan, implement and review loop, passed a local validation gate and published a pull request. The coordinator assessed each candidate, merged it when every gate was green, rebased the remaining branches and started the next wave of work. Ten days later, 183 pull requests had been merged. I could not sign in to a running version of the system I was building.

That gap, between an extraordinary volume of activity and the absence of anything I could touch, try and learn from, is the subject of this report. It is the wall that several of the most experienced practitioners in our field have been describing. Kent Beck compares coding agents to genies that grant wishes literally, warns that plausible code is not working code, and has written about the existential crisis that follows when the visible part of a craft can be produced by a machine [11, 15, 16, 73]. Robert C. Martin argues that agents will out-code us, so our discipline

*Microsoft, Zurich, Switzerland. Contact: `juanserv@microsoft.com`. The views expressed are the author's own and do not necessarily reflect those of Microsoft. The case organisation, Zava North Homes, is fictional.

must move into constraints that machines enforce [64, 65]. Andrew Murphy reminds engineering leaders that the speed of writing code was never the bottleneck [68]. I knew these warnings and followed many of them. I hit the wall anyway, and the way I hit it is instructive.

My working theory is simple. Software that only accumulates features, without deliberate consolidation, decays: it becomes more complex, more fragile and slower to change. That has been documented for decades [26, 32, 58, 77]. I propose that agentic development may run the same decay faster, through two possible accelerants. The first is compounding error. Each step builds on the agent's previous output, including its misreadings of intent and complexity nobody asked for; Beck's genie grants the wish as stated, not as meant [73], and every literal wish becomes the context for the next. The second is volume. Agents produce more code and documentation than people have the time or patience to review and test, so the decay goes unseen until something breaks. If the theory holds, the remedy is the one Beck describes: stop adding features blindly, and after each feature stop, review, refactor and improve before adding the next [12, 14, 16].

That raises the central question of this paper: can an additional agent perform some of the reflective work normally supplied by humans, recognise when development is drifting and steer it back towards a useful outcome? I use reflection to mean checking assumptions and a working model of the system against observed behaviour, not generating another approval record. I read Beck's alternation of features and futures as preserving options for later development: adding behaviour can narrow those options, while consolidation can restore them [14, 16]. Whether an agent can supply that corrective work is an empirical question, not something established by the fact that a human supplied it in this case.

The case is unusually well documented. The repository is public, every change went through a pull request, an append-only log recorded what the agents did and why, and the coordinator's own instructions are versioned with the code [84]. I approach the central question through four more specific questions:

- **RQ1.** What did a long-running, autonomous, spec-driven delivery loop produce, and what did it fail to produce?
- **RQ2.** Which proposed mechanisms fit the gap between activity and usable outcome, and do they fit the working theory?
- **RQ3.** Can an additional agent preserve correctness and future changeability through a consolidation step after every feature, and at what cost? The experiment tests this bounded form of correction, not general autonomous steering.
- **RQ4.** Which reflective responsibilities might be delegated to agents, and what evidence and human accountability would that require?

The report makes five contributions:

1. A documented case of one architect working with agents for six weeks, from desk research to a deployed MVP and then a ten-day autonomous delivery loop, with quantitative evidence from the repository, its issue tracker and its log (Sections 3 and 4).
2. Five proposed mechanisms consistent with the loop's activity without usable software, each grounded in the case, mapped onto the working theory, linked to prior work and paired with an early-warning signal (Section 5).
3. A feedback-first operating model for agentic delivery, made of invariants, checkpoints, proportional gates and measures that an architect can apply on the next working day (Section 7).
4. A consolidation harness and a controlled experiment protocol to test an intervention whose effect between feature steps is not isolated in the related work reviewed here [3, 71, 78]. The protocol follows the registered-report example of Lulla et al. [62] and HVE Core's Minimum

Viable Experiment method, with hypotheses and decision rules to reconcile with the tagged protocol before registration and fresh confirmatory runs (Sections 7.1 and 8).

5. A distinction between the reflective work I supplied in this case and the open question of how much of it an agent could perform (Section 9).

This is not an evaluation of the tools I used. The framework at the centre of the case keeps a human between its phases by default; the failures described here occurred in a configuration I designed that delegated delivery and merge decisions (Section 2.6). I still handled approvals and technical blockers, but no longer learned from using the replacement product. I cannot isolate that change from the others.

2 Background

2.1 Software that only grows

That software decays when it only accumulates features is one of the oldest observations in our field. Lehman's laws of software evolution hold that a program in real-world use must keep changing, and that its complexity grows as it evolves unless work is done to maintain or reduce it [58]; a later formulation adds that its quality will appear to decline unless it is rigorously maintained and adapted [59]. Parnas called the result software aging, and blamed in part the changes made by people who do not understand the original design [77]. Eick and colleagues found mixed but, on the whole, persuasive evidence of code decay in fifteen years of change data from a large telephone-switching system [32]. Cunningham's technical-debt metaphor framed the cost of not consolidating as interest paid on every later change [26], and Fowler's design stamina hypothesis holds that without attention to design, adding features eventually becomes slower than it would have been with it [34]. Studies have since put numbers on the interest: across 39 proprietary code bases, low-quality code had fifteen times more defects and took 124% longer to develop than high-quality code [91], and developers in a longitudinal study lost about 23% of their working time to technical debt [18].

The evidence has limits. Lehman's laws are conditional: they predict decay only when no counter-work is done. Increasing complexity and declining quality are among the least empirically supported of the laws, and some open-source systems, Linux among them, have grown without the predicted decay [45]. Several of the studies above are correlational. The common lesson is narrower and sturdier: quality is not preserved by default. It is the product of work that feature delivery tends to crowd out.

2.2 Why agents might accelerate the decay

Two properties of current agents could speed that decay up. The first is compounding error. When language models execute long tasks they condition on their own earlier mistakes, so errors become more likely as tasks lengthen [87]; in multi-turn, underspecified conversations their performance drops by 39% on average, largely because they commit to early, wrong assumptions and then build on them [53]. Coding benchmarks show the same pattern at the scale of a code base. In SlopCodeBench, where agents repeatedly extend their own solutions, structural erosion rises in 77% of trajectories, agent code is 2.3 times more verbose and twice as eroded as comparable open-source code, and explicit quality instructions improve the starting point without changing the rate of degradation [72]. In SWE-CI, which replays months of real repository history, regressions increased with iterations for most of the models tested [23]. All 20 models scored below human oracle code on the Maintainability Index, although 15 of 20 scored better on Pylint; most had a zero-regression rate below 0.25. Code could look clean on the surface while scoring worse on structure. Heander et al. found a related caution in agentic review: better CodeHealth was associated with lower token cost, but batch fixing left median CodeHealth

unchanged [44]. Given only a requirements document and an empty repository, agents lose global coherence [28], and some "pass" tests by modifying them [97]. Field data point the same way. Open-source projects that adopted an AI coding tool saw a short-lived jump in velocity but persistent increases of about 30% in static-analysis warnings and 41% in code complexity, which in turn slowed later development [43]; repository-scale analyses report copy-pasted code rising and moved, that is refactored, code falling as AI assistance spread [39]; and DORA associated a 25% increase in AI adoption with better documentation and code quality but lower delivery stability (−7.2%) in 2024, with stability still lower in its 2025 report, which describes AI as an amplifier of existing strengths and dysfunctions and recommends working in small batches and a user-centric focus [29–31, 42].

The evidence is not one-sided. A preregistered randomised trial found no significant difference in how easily developers evolved code that others had written with or without AI assistance [20]. A study of chained maintenance tasks found that the downstream cost of agent edits comes mainly from inherited test failures and disappears when the earlier tests pass, which points to regressions rather than complexity as the channel of compounding [76]. And the best-known controlled productivity result, experienced developers 19% slower with early-2025 tools while believing they had been 20% faster [17], has since been qualified by its authors, who think developers are probably faster now [66]. Its durable lesson is the gap between perceived and measured progress.

The second property is volume. Human review works best on small changes in short sessions: industrial guidance is to review no more than a few hundred lines at a time [25], the median change at Google is about two dozen lines [82], and understanding the change, more than finding defects, is the main challenge of modern code review [4]. Agents break those conditions. Among repeat reviewers of agent-authored pull requests, approval rates rose while inline comments fell, a pattern its authors call habituation [96]. Multi-agent systems add failure modes of their own, in system design, inter-agent misalignment and verification [22]; agent builders advise sharing full context rather than splitting it across agents that each see a fragment [95]; and models use long contexts unevenly and less reliably as they grow [46, 61]. By contrast, tests that clarify intent help both the model and the person judging its output [33].

2.3 Optimisation with behavioural feedback

Pavlenko et al.'s *Towards an AI Software Factory for Data Systems* offers a complementary case [78]. They describe an Amdahl's-law limit: faster coding alone yields limited improvement across the development lifecycle. Their response deliberately narrows the problem to *Evolutionary Coding Tasks*: existing, human-architected systems with stable semantics and automatically measurable objectives, such as latency. Hotspan uses telemetry, code analysis and past reviews to select optimisation targets and package evaluations. Darwin combines local code–test–debug–reflect cycles with agents that generate, refine and select hypotheses across candidate solutions. This is agent-led steering within an explicit objective, not merely faster execution of a fixed backlog.

The authors report improvements across 45 open-source systems, with a mean performance gain of 13.8% for the best-performing twenty, not the full set. For a Microsoft performance team, they report 0.35 pull requests per engineer-hour of application work, estimated at three times the rate with agentic coding tools. That calculation excludes the 57% of team time spent building infrastructure and onboarding repositories; the output enters engineering review queues. These are reported optimisation results and an application-work productivity estimate, not a controlled estimate of end-to-end delivery, maintainability or future changeability. General semantic equivalence, automated reviewing and privacy-preserving shadow execution remain open challenges in their account [78].

The contrast matters for this paper. It is evidence against treating high agent throughput as inherently unproductive: an explicit objective and behavioural feedback can support useful

improvement. It does not isolate the effect of consolidation between feature additions or establish that autonomous optimisation can choose the right product. The proposed experiment addresses that narrower consolidation question, not the invention of agent-led reflection or continuous optimisation.

2.4 The practitioner conversation

The closest practitioner argument is Waseem et al.'s account of internally developed MVPs and industry reports [92]. They describe the same trade-off: rapid generation can accumulate debt through architectural inconsistency, security vulnerabilities and maintenance overhead. Their account includes descriptive security-scan counts but does not estimate a controlled effect of consolidation. I do not claim that argument as new. I add a public case trace, proposed mechanisms with early-warning signals, and a falsifiable experiment protocol.

Kent Beck has been the most persistent chronicler of this shift. In 2023 he wrote that the value of most of his skills had dropped to zero while the leverage of the rest had multiplied [11]. His 2025 experiment in *augmented coding*, building a B+ tree library with an agent under strict test-driven development, separated it from vibe coding by whether the programmer still cares about the code, its complexity, the tests and their coverage. He named the warning signs he watched for: loops, functionality nobody asked for, and tests disabled or manipulated so that they pass [13]. He describes agents as genies that grant wishes literally rather than as intended, and that will delete tests and declare themselves finished [73]. In his Prodacity 2026 talk he argued that fully agentic work without feedback goes off the rails, that the best software comes from maximising learning with software as a side effect, that every feature burns options that must be restored by alternating features with consolidation, and that one-shot spec-driven development is "waterfall wearing a new coat" [14, 16]. In a September 2026 essay addressed to mathematicians facing AI systems that prove theorems, he separates a discipline's visible output from the less visible work of understanding, simplifying and building abstractions, and places lasting human value in the latter [15].

Robert C. Martin, who in 2024 dismissed doom-laden predictions about AI while treating language models as useful tools [63], now argues that agents will out-code humans by a wide margin. Programmers should therefore use that leverage to enforce test coverage, mutation testing and complexity limits through deterministic, executable standards, writing tests first and knowing their customers [64, 65]. Erich Gamma, discussing the open-sourcing of AI features in Visual Studio Code, made a related point about evaluation: once a language model is part of a test, "your test is, by definition, flaky", so his team built stochastic simulation tests instead [37]. Andrew Murphy applies the Theory of Constraints [40]: accelerating a non-bottleneck fills review queues and inflates work in progress, so that "you are producing more code and shipping less software" [68]. He has also described engineers' grief at the loss of their craft [67].

Voices from inside the HVE community and from Microsoft's product groups say similar things. Mike Lanzetta, who writes about hypervelocity engineering, advises early-career engineers to work in small chunks, test first and limit the complexity of files and functions, and says that experienced developers "should expect to spend more time in Pull Request reviews, not less" [55]. After pushing roughly 60,000 lines of code in a two-week sprint with a crew of specialised agents, he described re-explaining architectural constraints to agents that had drifted from the original intent, observed that in a high-throughput environment "quality drift creeps in fast", and responded with a mandatory done gate and sprint-boundary retrospectives whose findings block the next sprint until they are resolved [56]. Charles Lamanna reports that agentic coding moved his organisation's bottleneck from writing code to deciding what matters and knowing whether what was built is good. Rather than spending the surplus on more features, his teams shipped about as many as before with more depth, reliability, performance and security, and one product's support tickets fell by double-digit percentages year over year while usage grew. Counting inputs such as lines of code, he writes, "is easy and tells you almost nothing" [54].

Spec-driven development (SDD) is the most direct attempt to tame agents with structure. GitHub's Spec Kit proposes a specify, plan, tasks and implement flow in which the developer's role is to verify, not merely to steer [27]. Birgitta Böckeler's review of SDD tools distinguishes spec-first, spec-anchored and spec-as-source approaches, reports one tool turning a small bug into four user stories with sixteen acceptance criteria, observes that agents both ignored and over-followed instructions, and concludes: "I'd rather review code than all these markdown files" [19]. A study of SDD in practice reports more reliable agent behaviour but higher planning and review costs, and specifications that decay [1]. Other voices frame the same tension: *vibe coding* for throwaway projects [50]; *vibe engineering*, in which experienced engineers stay accountable [93]; the observation that AI gets a project most of the way while the rest needs engineering judgement [74]; software engineering joining the non-deterministic engineering disciplines [35]; and the advice to hold back on complacency with AI-generated code [90].

2.5 Older theory

None of this is new in kind. Peter Naur argued that programming is primarily the building of a theory, held by the programmers, of how the program solves its problem, and that documentation cannot fully carry that theory. In his example, a second team equipped with full documentation still made changes that destroyed the power and simplicity of the original design [70]. Brooks located the essential difficulty of software in specifying, designing and testing the conceptual construct rather than in representing it [21]. Royce, often cited as the origin of the waterfall model, called the purely sequential approach risky because testing at the end is the first time behaviour is experienced rather than analysed [80]. Gall observed that complex systems that work evolve from simple systems that worked [36]. Extreme Programming built its practices around short feedback loops and small releases [8, 9]; Cockburn's walking skeleton and Hunt and Thomas's tracer bullets both start with a thin end-to-end implementation [24, 48]; and the Agile Manifesto values "working software over comprehensive documentation" [2]. Beck's *Tidy First?* treats design as small, reversible structural changes made before behaviour changes [12].

From human factors, Bainbridge's ironies of automation explain why automating the routine parts of a task leaves operators with the hardest parts, monitoring and intervention, while eroding the skills those parts need [5]. Recent work applies this to generative AI: users shift from producing to evaluating, workflows are restructured unhelpfully, and automation makes easy tasks easier and hard tasks harder [86]. Working with generative AI makes metacognitive demands similar to delegating to a team [89]; higher confidence in AI is associated with less critical thinking [57]; and AI that challenges rather than obeys has been proposed as a counterweight [83]. Finally, Goodhart's law, in Strathern's formulation, warns that a measure used as a target "ceases to be a good measure" [41, 88].

2.6 HyperVelocity Engineering (HVE Core)

HVE Core is an open-source framework from Microsoft that describes itself as an opinionated agentic software development lifecycle built on GitHub Copilot, guided by the principle "AI carries the rules, humans keep the judgment" [49]. Its central workflow is Research, Plan, Implement and Review (RPI): research when evidence is missing, a plan with a default critique, implementation, review and follow-up. Each phase writes dated artefacts under a `.copilot-tracking` folder, so that context travels through files rather than chat history, and the framework treats these artefacts as git-ignored working memory by default. Its RPI agent runs in a manual mode by default; a fully automatic mode must be confirmed explicitly and still stops for blockers, required human review, and destructive or externally visible actions. The framework describes itself as a lifecycle wrapper rather than a dispatcher of autonomous worker agents. Around RPI it offers product-side agents: a Design Thinking coach, builders for business and product requirements documents (BRD and PRD), a Socratic assistant for architecture decision records (ADRs) and

a backlog manager for GitHub, Azure DevOps and Jira. The project warns that it evolves quickly and is best treated as a source of patterns [49]. Its experimental collection includes an Experiment Designer agent that coaches a team through a Minimum Viable Experiment: falsifiable hypotheses with success thresholds set before any run, vetting against red flags, a minimal design, and an evaluation of the outcome against the precommitted criteria [49]. Section 8 uses it. I used the HVE Core plugin snapshot `plugins-v3.3.106` throughout the case. Its snapshot commit is `2df5013e29ebfd2ce61bcd0cf021d8c8186482eb`, generated from source commit `e9af018fe5ffbfbc72321a8bca21fb987359af08`. Historical use is my account; the snapshot identifiers do not establish an installation date.

3 Case and method

3.1 Setting

The case is my capstone for Microsoft's *Azure Master Architect* (AMA) programme. Its name is public in the Microsoft Employee Learning badge listing.[1] That public listing does not authorise publication of internal assessment material or this paper. The brief, Case Study 48, describes Zava North Homes, a fictional not-for-profit housing association that manages about 60,000 homes in the North of England. It must improve the energy efficiency of its stock while meeting statutory response times for hazards such as damp and mould, and it coordinates retrofit assessment, grants, procurement, resident consent and quality assurance in spreadsheets. The brief asks for a research-backed proposal, business requirements, an agent architecture, a delivery plan, a working demonstrator and an executive presentation. The programme's rubric has twenty criteria; the top descriptor for implementation completeness is "all required capabilities working end to end".

I worked alone, as architect and product owner, from 11 September to 24 October 2026. Agents did almost all of the research synthesis, writing and coding. The repository also contains an automated scorer: a GitHub agentic workflow that asks a language model to score each pull request against the rubric and records the result [84].

3.2 Workflow and tools

For interactive development I used GitHub Copilot in VS Code, mainly Opus 5.5 during the first two weeks and then GPT-6.1 Sol through the case cut-off. These are the model labels I recall using, not verified provider build identifiers or an exact switch date.

- **Research.** Sixteen official legislation, regulator, policy, standards and funding documents were downloaded, converted to Markdown and organised, with agent-written sector and company research, into a knowledge base: an index, immutable sources, maintained syntheses, an owner-curated memory file and an append-only log of every change.
- **Challenge.** Eight agents role-playing the case organisation's stakeholders (asset management, programme delivery, finance, procurement, data, building safety, neighbourhood housing and resident engagement) and HVE Core's Design Thinking coach challenged each idea. Their first verdict was that none of the five end-to-end agents requested in the brief should be built as such.
- **Requirements and decisions.** HVE Core's BRD and PRD builders, a BRD quality reviewer and the ADR assistant produced and revised the requirements and six architecture decision records.
- **Prototypes and MVP.** Several prototypes were built, reviewed by the role agents and kept as evidence. An MVP followed and was deployed to Azure.

[1] https://www.credly.com/org/microsoft-employee-learning/badge/azure-master-architect

- **Backlog.** HVE Core's backlog manager turned the requirements and decisions into GitHub issues with epics, features, acceptance criteria and dependencies, labelled by how much of each item already existed in the MVP.
- **Coordinator.** On 15 October I added a coordinator agent of my own design to deliver the backlog (Section 3.3).

3.3 The coordinator

The coordinator ran through Copilot CLI on a VM. It selected waves of issues whose dependencies were merged and launched parallel worker CLI sessions through `tmux`, each in an isolated git worktree with HVE Core's RPI agent and one issue as its brief. I used Sonnet 5.5 for the workers, including before the explicit pin. At the pinned repository commit the worker selector is `claude-sonnet-5.5`; PR #420 introduced that requirement on 21 October. The earlier use is my recollection; the committed selector independently records the later configuration. The coordinator used Auto selection with the Intelligence setting. I observed it tending to select GPT-6.1 Sol, but this was not a guaranteed model pin or proof of every session's actual model or provider build. Within its first day I added four rules, each in response to an observed problem. First, it could merge its own pull requests through a gate it verified itself, because repository auto-merge was unavailable. Second, the full research, plan, implement and review loop became mandatory for every issue, and the presence of the loop's `.copilot-tracking` artefacts in the pull request became a merge condition, because the first waves of sessions had implemented directly without them. Third, it updated the project board and ticked only the acceptance criteria that reviewed evidence proved. Fourth, it monitored sessions on a schedule instead of blocking. Later rules required a complete local validation gate before every push, including documentation-only changes, covering five source projects, infrastructure templates, a disposable SQL Server, browser accessibility checks, migrations, four container images, vulnerability scans and a software bill of materials, to be rerun after every correction or rebase. An issue stopped only on a *human blocker*: an owner decision, a named approval, a missing credential, or a review finding still open after two correction cycles. On the same day an accepted ADR made migrating the working MVP into a new source layout the first priority and barred new feature code from the MVP.

3.4 Data and analysis

I use an exploratory single-case design, with myself as participant-observer. The case is the delivery of this repository from 11 September to 24 October 2026. The unit of analysis is the autonomous loop of 15–24 October. The MVP period of 8–14 October, when I used the system intermittently to refine ideas and the user experience, provides a within-case contrast, not a control group. I chose the case because it is revelatory: a public, contemporaneous trace of an autonomous agent loop.

I analysed the public repository at commit `29c6b51` (24 October 2026, 16:19 UTC) [84] with issue and pull-request metadata retrieved from the GitHub API on the same day. The main sources are the append-only knowledge-base log (368 dated entries and about 57,000 words from 11 September to 22 October; entries before 25 September were reconstructed from history), the knowledge-base index, the BRD (version 0.19), the PRD (version 0.18), the coordinator's definition, issue bodies and the rubric scorer's latest result. I used descriptive counts; a hedging lexicon applied to each log entry (Appendix A); reference tracing, in which I followed the identifiers of recorded learnings through the log by date; and AI-assisted interpretation of my observations. I described the mechanisms, the proposed accelerants and the consolidation remedy. GitHub Copilot in VS Code organised those explanations, named the five patterns and drafted the account, using Opus 5.5 and then GPT Sol 6 and 6.1 as reported interface labels. Microsoft 365 Copilot helped summarise material and find relevant reference documents; it did not perform

Table 1: Timeline of the case (all dates 2026).

Dates	Phase	What happened
11 Sep – 7 Oct	Research and requirements	Sixteen regulatory sources converted; role agents reject five end-to-end agents (26 Sep); BRD and PRD drafted and repeatedly revised; prototypes reviewed; my usability feedback recorded (7 Oct).
8 – 14 Oct	A usable MVP	MVP built and reshaped through several feedback passes in one day (8 Oct); running on Azure with SQL, blob storage and two replicas over 980 synthetic cases (9 Oct); rebuilt with Entra ID sign-in, which I used (12–13 Oct); five assistant paths exercised against a real model (14 Oct).
14 – 15 Oct	Backlog and coordinator	Backlog generated as GitHub issues; coordinator created and hardened within a day; an ADR makes migrating the MVP to a new source layout the first priority.
15 – 22 Oct	The loop at scale	Pull requests merged at about eighteen a day; the log grows by about 37,000 words; the migrated deployment holds one fictional case, no staff and no model deployment (16 Oct); browser sign-in blocked; validation moved off hosted CI to save budget (18 Oct); cloud cost ceiling raised (22 Oct); new cross-cutting requirements appended to open issues (22 Oct).
22 – 24 Oct	Asking for something that runs	Ten issues labelled *feedback-first*; instruction to make employee and resident sites available as soon as possible with at most eight parallel items; checkpoint and refactoring instruction (24 Oct).

the log analysis or develop the mechanisms. This was not a systematic entry-by-entry qualitative coding study. I reviewed important specification sections selectively, especially the addendum, rather than auditing the full BRD and PRD. Their identifiers and cross-references made the overall design difficult to follow. I have checked the repository figures and Azure expenditure reported here. Independent author checks of numerical findings from cited studies and of the log-growth figure remain pending. The report follows established guidance for case study research in software engineering [81] and the ACM SIGSOFT Empirical Standards [85]. As the sole developer and author I am a participant-observer; Section 10 discusses what that implies.

4 What happened

Table 1 summarises the case and Table 2 compares the periods before and during the delivery loop.

4.1 Research, requirements and a usable MVP

Table 3 adds code-level activity from the pinned history. The script classifies paths and compares each selected first-parent commit with its first parent, detecting renames. The integration view includes multi-parent merges and PR-numbered squash commits: 31 before the loop and 177 during it. These are not the API pull-request counts in Table 2; the literal-merge subset, seven in each period, is also reported in `anc/repo_history_metrics.out.txt`. Classification rules and commands are printed there. The additions and removals show work volume, not net code size, maintainability or delivered value. No complexity comparison was made because `lizard` was not installed.

The September work was research and requirements preparation, not application building or a project-free baseline. The agents assembled a regulatory corpus, sector baselines and a modelled customer profile, and the role agents produced a genuinely useful early verdict: each

Table 2: Before and during the delivery loop. Counts from the repository at commit `29c6b51` and the GitHub API on 24 October 2026; hedging phrases as defined in Appendix A.

Measure	Before the loop	During the loop
Pull requests merged	52 (26 Sep – 14 Oct; 2.7 a day)	183 (15 – 24 Oct; 18.3 a day)
Knowledge-base log entries	126 (to 14 Oct)	242 (15 – 22 Oct)
Words appended to the log	19,967 to 14 Oct (about 1,000 a day from 25 Sep)	36,755 (15 – 22 Oct; about 4,600 a day)
Hedging phrases per 1,000 log words	1.2	3.8
Log entries citing my simplicity requirements	4 (last on 13 Oct)	0
Deployment I could use	Entra sign-in, 980 synthetic cases, live model calls (12 – 14 Oct)	One fictional case, no staff, no model deployment (16 Oct); sign-in blocked

Table 3: Text lines added and removed in first-parent integrations, including PR-numbered squash commits, through `29c6b51`. Detected pure renames add no lines; binary deltas are excluded.

Path class	8–14 Oct		15–24 Oct	
	Added	Removed	Added	Removed
Product source	8,136	421	23,408	2,184
Tests	6,871	537	73,478	1,416
Documentation	12,065	1,297	25,210	2,455
Tracking artefacts	238,185	2,642	240,369	1
Infrastructure	2,708	73	6,162	485
CI and workflows	240	64	897	246
Other	25,320	1,099	72,711	1,776

requested *end-to-end* agent bundled deterministic controls, accountable human decisions and a smaller set of tasks suited to bounded AI, so none should be built as such. Requirements were written, reviewed by an automated quality reviewer and revised many times. Application prototypes followed in October.

On 7 October I used the latest prototype myself and recorded what was wrong with it. The next day the agents combined my notes with critiques of three prototypes by the UX, Design Thinking and role agents into ten simplicity requirements (SA-001 to SA-010): a default view listing only my cases, each with one plain status and one next action; a case that opens full screen; blockers explained in plain words with an owner; no role simulator in user views; optional AI that never blocks manual work; and so on. On 8 October they built an MVP that followed those requirements and then reshaped it through several rounds of my feedback the same day: filters, an overview map, visits, handover suggestions and document uploads. On 9 October it ran on Azure with Azure SQL, blob storage and two replicas over 980 synthetic cases. On 12 October the stack was rebuilt from an empty resource group with Entra ID sign-in; I signed in and used it, and the problems I reported, a missing logo and an account switch that only signed me out, were fixed within a day. On 14 October five assistant paths produced cited proposals through a real model deployment, using 6,884 tokens across five calls. I tested the initial proof of concept for a couple of days, discarded it, then used the MVP for a few days to refine ideas and the user experience. This was intermittent use, not daily use throughout the week. I then carried those ideas into requirements, ADRs and the hierarchical backlog.

I treat 8–14 October as a within-case contrast with 15–24 October: intermittent use of a usable MVP versus no realistic end-to-end task completed in the replacement during the autonomous

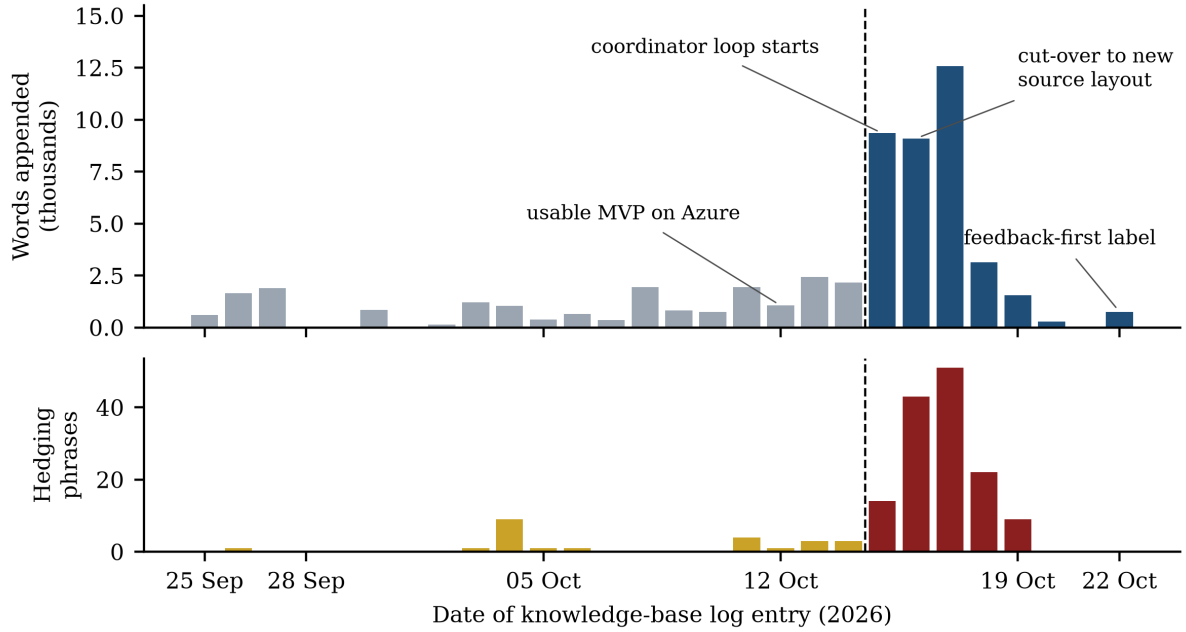


Figure 1: Words appended to the knowledge-base log per day (top) and hedging phrases per day (bottom). The dashed line marks the start of the coordinator loop on 15 October 2026.

loop. I remained involved, mainly answering approval requests and resolving technical blockers to keep the loop running. The contrast concerns product-use feedback, not the complete removal of human oversight. It is consistent with the importance of feedback, but cannot establish that removing product-use feedback caused the outcome: the migration, workload, gate rules, models and deployment conditions also changed, and the periods were not assigned at random.

4.2 The backlog and the coordinator

Late on 14 October the backlog manager generated the backlog from the BRD, PRD and ADRs. Many issues carried a label stating that part of the work already existed in the MVP. On 15 October I created the coordinator and, as Section 3.3 describes, hardened it within the day. The log recorded 91 entries that day, against at most 19 on any earlier day. The accepted source-layout ADR made the migration of the MVP's code the first priority; the MVP itself was to be retired.

4.3 The loop at scale

From 15 to 24 October, 183 pull requests were merged, compared with 52 in the nineteen days before. The knowledge-base log grew by 36,755 words between 15 and 22 October, 31,005 of them in the first three days (Figure 1). The share of hedged statements, such as "remains unmet", "not inferred" or "not accepted", rose from 1.2 to 3.8 per thousand words.

The migration succeeded on its own terms. A private deployment built from the new source layout passed health checks, rejected unauthenticated requests and survived restarts. But it held one fictional case, no staff accounts and no model deployment, against the 980 cases, eleven staff identities and five working assistants of the system it replaced. A public entry point was restored through an additional edge service, yet real-user sign-in stayed blocked because the new deployment had neither an application registration nor staff bindings, the very things set up and verified on 12 October. Recovering them became a chain of issues, each with its own research, plans, reviews and corrections. To preserve the hosted CI budget, validation moved to a local gate on 18 October (issue #413). The demonstrator's cloud resources accrued USD 189.94

between 1 and 20 October, with a forecast above the USD 500 ceiling I had set; on 22 October I raised the ceiling to USD 700, deleted a test edge profile and scaled a demo service to zero. The same day, after a mentor raised the economics of AI consumption, the agents produced new BRD and PRD versions and appended a new set of acceptance criteria to fifteen open issues.

A Copilot usage dashboard captured on 24 October showed USD 1,809 of price-list consumption value, 10,229,625,825 tokens and 100,938 requests in the rolling last 28 days. These were account-wide totals across all work, not AMA-only measurements or an invoice. The token total included 10,081,179,684 cached tokens. Surface and model breakdowns described the same total and must not be added together; the requests were not established as premium requests. The screenshots do not isolate 15–24 October, establish exact window boundaries or support an estimate of the loop's incremental cost.

4.4 Asking for something that runs

On 22 October I asked the loop to prioritise something I could use. A *feedback-first* label, described as prioritising "the bounded usable synthetic demo slice", was applied to the relevant issues, ten of them by 24 October. On 24 October I asked for employee and resident sites to be available as soon as possible, with at most eight items in parallel. These instruction dates and the eight-item cap are my confirmed recollection, not quotations recovered from the pinned log. The agents created the smallest independently runnable resident preview they could design (issue #444): read-only, synthetic and bound to the local machine, with thirteen acceptance criteria. The same day a two-line fix to the rubric scorer arrived with a 78-line evidence record and an explicit, one-time owner exception to interrupt the full local gate (pull request #445). Following Beck's advice to alternate features with consolidation, I also instructed the coordinator to stop at checkpoints, review what had been built, exercise the functionality and refactor. I do not evaluate the outcome of that instruction here.

5 Five mechanisms

I propose five mechanisms consistent with how a loop that merged a pull request every eighty minutes or so left its owner with nothing to use. Table 5 summarises them.

Read against the working theory of Section 1, they fall into groups (Table 4). The base rate is feature accumulation without consolidation: until 24 October the loop had no step that paused feature work to review and refactor what had been built, and its one large restructuring, the migration to a new source layout, was planned up front as backlog items rather than done continuously. Specification inflation, proxy capture and learning evaporation are forms of the first accelerant, compounding error: each misreading, or literal reading, of intent became the context, the criteria or the evidence for the next step. Process-gate inflation and feedback starvation may reflect the second, unreviewable volume: gates and caveats coexisted with the absence of a person using the product.

5.1 Specification inflation

The specification grew faster than any system could absorb it or any person could validate it. By 22 October the BRD had reached version 0.19 and about 37,900 words, with requirement identifiers up to FR-052 and AC-103; the PRD had reached version 0.18 and about 33,900 words, with identifiers up to FR-061 and AC-110. The backlog inherited that scale and grew to 192 issues. Cross-cutting concerns were appended to existing issues rather than sequenced as their own work: a responsible-AI section on 15 October and an AI-consumption section on 22 October. Issue #126, on pilot release gates, ended with seventeen acceptance criteria in three sections. Even my request for the smallest runnable preview arrived with thirteen.

Table 4: A working theory: a base rate of decay and two accelerants, with the mechanisms that instantiate them in this case.

Driver	What it predicts	In this case
Base rate: features accumulate without consolidation	Complexity grows and quality declines unless work is done to counter them [58, 59]	No consolidation step until 24 Oct; restructuring planned as backlog items
Accelerant 1: compounding error	Each step builds on earlier misreadings and unneeded complexity [53, 72, 87]	Specification inflation; proxy capture; learning evaporation
Accelerant 2: unreviewable volume	Output exceeds what people can review and test, so decay goes unseen [25, 96]	Process-gate inflation; feedback starvation

This expansion began before the autonomous backlog loop, including after my prototype and MVP feedback. The requirements agents sought fully specified BRDs and PRDs, named approvers and resolved ownership before building. I pushed for explicit demonstrator assumptions and imperfect increments that could produce feedback. Smaller, iterative requirements documents are a plausible alternative, not an intervention tested in this case.

This is Böckeler's small bug that became sixteen acceptance criteria [19], at the scale of a programme. It is also what Royce and Gall warned about: a large system specified up front and tested at the end [36, 80]. Brooks explains why more text does not help: the essential difficulty lies in the conceptual construct, and writing it down in more places does not reduce it [21]. The benchmark evidence points the same way: agents given only a requirements document lose global coherence [28]. Beck's metaphor fits: every feature, and every acceptance criterion, burns options [14, 16]. The early-warning signal is the ratio of specification growth to behaviour a human has validated in the same period. Here the denominator was close to zero.

5.2 Proxy capture

In this case, artefacts made into gates appeared to displace outcomes. I interpret three patterns this way.

Merged pull requests are a weak proxy for delivered value. Murphy-Hill et al. report roughly 24% more merges among adopters in Microsoft's rollout, while explicitly distinguishing merges from value [69]. Peralta et al. studied 9,799 human-reviewed agent pull requests; in their manually inspected samples, only 35.7% of rejections reflected clear agent failures and 15.4% of merges required explicit reviewer involvement [79]. AIDev shows the scale of this measurement problem: 932,791 agent-authored pull requests from five agents across 116,211 repositories, not a count of delivered outcomes [60].

Process. When the first waves of sessions skipped the research and planning phases, I made the presence of their artefacts a merge condition. From then on every pull request carried research, plan, change and review records; the artefacts were guaranteed, the thinking they were meant to evidence was not. Pull request #445 changed two lines of behaviour and added a 78-line evidence record that reports its own artefact completion as 100%. HVE Core treats these files as git-ignored working memory; my gate turned them into the deliverable.

In the first-parent integration view, tracking-artefact additions were 10.27 times product-source additions during the loop, compared with 29.28 before it (`anc/repo_history_metrics.out.txt`). This supports concern about artefact volume, not a claim that the ratio rose when autonomy began. Churn alone does not establish code decay.

Evaluation. The rubric scorer, a language model reading the repository, rated implementation completeness at five out of five, "all required capabilities working end to end", citing a README and a check script, at a time when I could not sign in to the system. Gamma's remark about tests applies to judges too: put a model in the evaluation and the evaluation becomes stochastic [37], and a judge that reads the repository is scoring the text that agents are best at producing.

Honesty. I had instructed the agents never to fabricate approvals or overstate evidence. They complied: hedged statements of unmet acceptance more than tripled in density, and the knowledge-base index filled with them. Issue #349 records that a browser redirect "is not successful sign-in". The agents were right, and still nobody could sign in. Honest boundary statements became the output.

Goodhart and Strathern predict the first two levels [41, 88]; Beck observed agents deleting tests to finish [73], and ImpossibleBench measured it [97]. The METR study adds the human side: the feeling of progress is an unreliable instrument [17]. The early-warning signals are the ratio of evidence artefacts to behaviour changes, the share of merged pull requests that change what a user can do, hedge density, and any divergence between automated scores and a live demo.

5.3 Learning evaporation

Some recorded lessons appeared to fall off the loop's read path, while important design decisions survived. Maintaining the structured log and memory became difficult as the record grew. I observed drift, not a complete loss of the project's theory. Before the loop, my use of the MVP produced concrete lessons: the simplicity requirements, a deliberately public development profile for throwaway stacks, and the callback, role and staff-binding steps that made sign-in work. All were recorded, in the PRD, in a simplicity review and in infrastructure runbooks. The coordinator's briefs carried each issue's text, acceptance criteria and the repository conventions, but not those lessons. The log cites the simplicity requirements for the last time on 13 October; the development profile appears only on 8 October. The migrated deployment lacked the registration and staff bindings established on 12 October, and sign-in was blocked.

The read path itself decayed. The knowledge-base index, which every agent is told to read first, accumulated 1,069 words of issue-specific status notes ahead of its first instruction. The index, the log, the BRD and the PRD, the documents that define the project's state and intent, total about 139,000 words, or roughly 180,000 tokens at a typical ratio of 1.3 tokens per word: more than any agent can attend to reliably, even when it fits in its context window [46, 61].

Naur explains why recording was not enough. A program's theory lives in the people who build it, and a second team with full documentation can still destroy a design's simplicity [70]. In an agent loop, every session is a second team. Advice to share full context between agents [95] and the failure modes catalogued for multi-agent systems [22] describe the same risk. HVE Core's choice to carry context in files on disk is necessary but not sufficient: the files must be on the read path, short enough to read and, ideally, executable. The early-warning signals are lessons that exist only as prose, a read path that grows faster than the code, and problems that have to be solved twice.

Pavlenko et al.'s World Model offers a different design: preserve raw evidence, derive rebuildable views and retrieve compact, task-specific context with provenance and access controls. Updates are reviewed, and measured outcomes feed later decisions [78]. This distinguishes storing lessons from making them usable in the next decision; a small read path need not mean discarding history. It is an architectural alternative to an accumulating log, not a tested remedy for learning evaporation in this case.

5.4 Process-gate inflation

Each failure was answered with another gate, until governing a change cost more than making it. Gates are cheap for an agent to write and expensive for everyone to run. Within ten days the coordinator required, for every issue, the full RPI loop, a complete local gate before every push and after every correction or rebase, independent review, up to two correction cycles, board updates and evidence-based ticking of acceptance criteria. Hosted CI minutes had to be

rationed, the full gate had to be interrupted by explicit exception for a two-line fix, and paid cloud validation stayed blocked even after the cost ceiling was raised.

The scarcest resource was not compute but my attention. Eighteen merged pull requests a day is more than any person can review in depth, so I delegated merging to the coordinator. I still answered approvals and technical blockers, but stopped getting feedback from using a replacement product. RPI critiques and reviews addressed the current issue rather than the cumulative system; there was no regular phase for reflective consolidation or generalisation across completed issues. This is Bainbridge's irony in miniature [5]: the automation took the part of the work I learned most from, using the product, and left me the part people are worst at, monitoring a stream of plausible, well-documented changes. It matches the shift from producing to evaluating reported for generative AI [86], the delegation demands it creates [89] and the erosion of critical scrutiny that confidence in AI brings [57]. Murphy's diagnosis applies directly: the loop accelerated a non-bottleneck and filled every queue behind it [40, 68]. Martin is right that discipline must move into deterministic gates [64]; this case adds that the gates must check the product's behaviour rather than the process's paperwork. The early-warning signals are gate time per change, human review minutes per merged pull request, work in progress, and cost per merged pull request compared with cost per accepted outcome.

5.5 Feedback starvation

The loop optimised for completing a specified plan, not for shortening the path to the next piece of human feedback from a running increment. This is my proposed central mechanism, not an isolated cause. Nothing in the coordinator kept a usable system alive: no invariant said that after every merge I must still be able to sign in and do my work. Done meant artefacts, criteria and gates. In the MVP week the feedback loop included my intermittent use, with some changes made the same day. From 15 October the first priority was restructuring code, the working MVP was frozen, and its replacement had not become usable by the time of the case cut-off. When I finally asked for something that runs, the request was absorbed by the same machinery: a label, new issues, thirteen acceptance criteria and the full set of gates.

Every older practice I cited in Section 2.5 exists to prevent exactly this: short releases and test-first development [8, 9], the walking skeleton and tracer bullets [24, 48], and working software over comprehensive documentation [2]. Beck's warning that fully agentic work without feedback goes off the rails [16], DORA's emphasis on small batches and user focus [30] and Murphy's list of real bottlenecks, which includes missing feedback [68], describe the same failure. The early-warning signals are the time from the start of a cycle to the first human use of a running increment, and the age of the last increment a human has verified as usable. At the time of writing the latter was at least ten days.

5.6 How the mechanisms reinforce each other

I interpret the mechanisms as a possible reinforcing loop (Figure 2). In this case a large specification accompanied a large backlog and parallel sessions. Artefact-based gates generated more documents; some recorded lessons were absent from later briefs; regressions were not exposed through routine product-use feedback. I responded to gaps with more rules, criteria and gates. That sequence is consistent with reinforcement, but does not establish each causal link. I propose a running increment and a human checkpoint as conditions of every cycle, to test whether they interrupt it.

6 Informal field observations

In October 2026 I presented HVE Core to two partners and discussed agentic delivery with peers. I omit partner names and generalise identifying details. At one partner session, an engineer

Table 5: Five mechanisms, their evidence in this case, the prior work that anticipated them and an early-warning signal for each.

Mechanism	Evidence in this case	Anticipated by	Early-warning signal
Specification inflation	BRD ≈37,900 and PRD ≈33,900 words; 192 issues; 17 criteria on one issue and 13 on the smallest preview	Royce; Gall; Brooks; Böckeler; Beck	Specification growth versus human-validated behaviour
Proxy capture	Artefacts made a merge gate; 78-line evidence record for a two-line fix; LLM judge scores completeness 5/5 with no usable system; hedge density tripled	Goodhart; Strathern; Beck's genie; ImpossibleBench; METR	Artefacts per behaviour change; scores that disagree with a live demo
Learning evaporation	Simplicity and sign-in lessons absent from briefs; last cited 13 Oct; read path ≈139,000 words	Naur; context studies; multi-agent failure taxonomy	Lessons that exist only as prose; problems solved twice
Process-gate inflation	Full gate on every push; CI minutes rationed; exception for a two-line fix; 18 merges a day delegated	Bainbridge; ironies of generative AI; Murphy; Goldratt	Gate time per change; review minutes per merge
Feedback starvation	Working MVP frozen; replacement unusable after ten days; request for a demo turned into more issues	XP; walking skeleton; tracer bullets; Agile Manifesto; DORA	Time to first human use; age of last usable increment

challenged the overhead of multi-agent orchestration, its token cost, how rules are enforced rather than merely stated, documentation bloat and the practice of committing agent tracking files, and asked how the approach compared with lighter alternatives such as Spec Kit. At the other session, concerns included trust in agent output and the net gain once costs are counted. And a fellow architect in my programme, looking at my backlog, asked how, with such a backlog, one learns how the solution works: Naur's question in one sentence. These are informal observations, not a systematically collected dataset. They suggest similar concerns, not independent confirmation of the mechanisms or their prevalence.

7 Feedback-first agentic delivery

The remedy is not fewer agents or less rigour. It is to put the rigour where the feedback is. Seven practices follow from the mechanisms; Table 6 lists what each means in concrete terms and what to watch.

P1. A running increment is an invariant, not a backlog item. Before generating a backlog, build a walking skeleton that a human uses, and make "it still runs and I can still do my job in it" a merge condition checked by a scripted user journey on the real deployment. Never freeze a working system before its replacement is usable.

P2. Small batches, a short horizon and capped work in progress. Generate a backlog for the next week or two, not for the whole product. Cap parallel sessions and the number of acceptance criteria per issue, and turn cross-cutting concerns into their own sequenced issues instead of appending them everywhere.

P3. Human checkpoints on a fixed cadence. Stop the loop every few merges or every day for a demo. Decide what to keep, cut or tidy, and do not start the next wave without a recorded decision. Alternate feature waves with tidying waves, as Beck recommends [12, 16].

P4. Executable learnings and a small read path. Turn each lesson into a test, a lint rule or an architecture fitness function within a day, and keep a one-page set of standing rules in every brief. Cap the size of the documents agents must read, archive old log entries, and keep agent working memory out of the main branch, as HVE Core does by default.

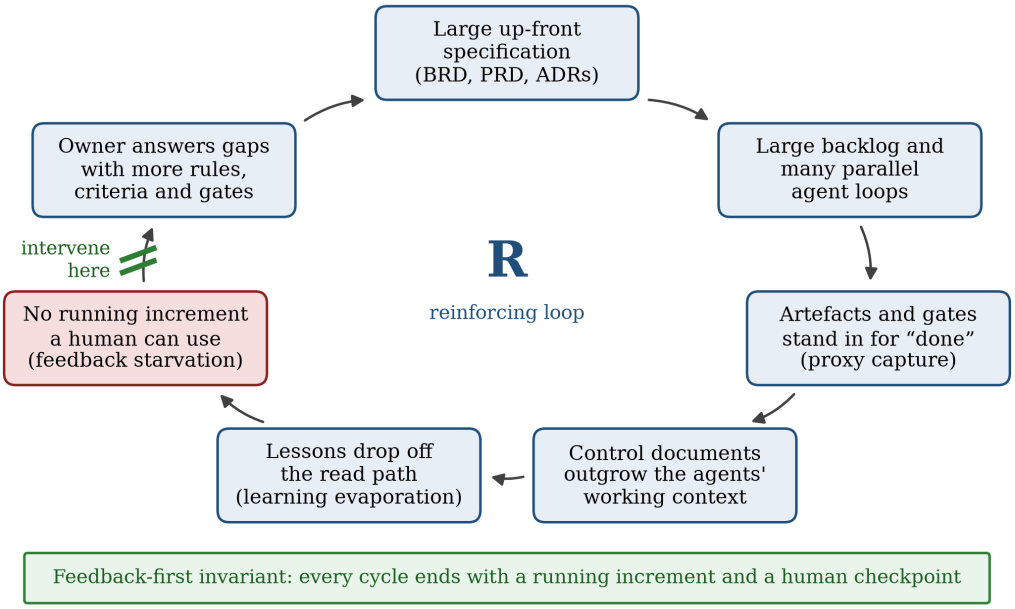


Figure 2: A proposed reinforcing loop consistent with the case, and a possible intervention through a feedback-first invariant.

P5. Proportional gates that check behaviour. Run change-scoped checks on every push and the full gate on merges to main or nightly. Gate on what the product does, not on the presence of process artefacts. Require no ceremony for trivial changes.

P6. Measure outcomes and treat automated scores as hypotheses. Track time to first human use, the age of the last usable increment, the share of merges that change user-visible behaviour, hedge density, review minutes per merge and cost per accepted outcome. Confirm any LLM-judged score with a live demo before believing it.

P7. Keep the human as the theory builder. Pair with the agent on the first vertical slice; write or approve the acceptance examples that define behaviour [33]; read the code of the core domain. Use agents that challenge as well as execute [83].

7.1 A consolidation harness

The idea is already practised. OpenAI describes replacing weekly manual clean-ups with recurring background agents that open small refactoring pull requests, alongside a documentation-gardening agent [71]. Anthropic asks each session to leave a clean state and update a progress file [3]. Neither account reports a controlled comparison of feature-only delivery with consolidation between feature steps. Darwin's continuous optimisation is a further precedent for automated improvement, but its performance evaluations do not isolate that intervention or test an unforeseen change request [78] (Section 2.3). Section 8 specifies the narrower comparison; it does not report results or claim a systematic review of every possible precedent.

Practices P3 and P5 suggest a narrower intervention that an agent can run inside the loop, leaving human checkpoints to a coarser cadence, and that can therefore test a bounded part of the central question: a consolidation harness. The pause is scheduled by the harness, not chosen by the agent; the agent is not authorised to reject features or redirect the product. After every feature the loop stops, and an agent:

1. runs the tests and a metrics script that lists complexity hotspots, duplicated blocks and lint warnings;

Table 6: Feedback-first practices: what they mean in concrete terms, which mechanism they address and what to watch.

Practice	In concrete terms	Addresses	Watch
P1 Running increment as invariant	Walking skeleton before the backlog; a scripted user journey on the real deployment gates every merge	Feedback starvation	Time to first human use; age of last usable increment
P2 Small batches	One to two weeks of backlog; at most three parallel sessions; at most five acceptance criteria per issue	Specification inflation; gate inflation	Work in progress; criteria per issue
P3 Human checkpoints	A demo every few merges or daily; keep, cut or tidy; tidying waves	Feedback starvation; specification inflation	Decisions recorded per checkpoint
P4 Executable learnings	Lessons become tests, lint rules or fitness functions; one-page standing rules; capped read path	Learning evaporation	Share of lessons that are executable; read-path size
P5 Proportional gates	Change-scoped checks per push; full gate on merge or nightly; no artefact gates	Gate inflation; proxy capture	Gate time per change
P6 Outcome measures	Behaviour-changing merges; hedge density; review minutes; cost per accepted outcome; demo before score	Proxy capture	Score–demo disagreement
P7 Human theory builder	Pair on the first slice; human-owned acceptance examples; read the core domain	Learning evaporation; feedback starvation	Questions the owner can answer without the agent

2. reviews the last feature's diff and the hotspots against a short checklist (duplication, long or complex functions, unclear names, dead code, rules from one feature leaking into another, tests that check implementation rather than behaviour);
3. refactors in small, behaviour-preserving steps, each applied through `test && commit || revert` [10], so that every change either keeps all tests green or disappears;
4. may add tests that pin current behaviour, but may not change or delete the example tests that define the features;
5. rewrites a design note of at most 300 words that says where each rule lives and what debt remains, replacing the previous one rather than appending to it;
6. stops when a quality budget is met or its time budget runs out.

Only then may the next feature start. Appendix B gives the prompt.

The harness turns Beck's alternation of features and futures [14, 16] and *Tidy First?* [12] into a gate, and follows Martin's advice to put discipline into deterministic checks [64]. It applies to every feature what Lanzetta's sprint-boundary retrospectives apply to every sprint [56], and it spends part of the agentic surplus on quality, as Lamanna's teams did [54]. It differs from the gates of Section 5.4 in what it checks: the behaviour and shape of the code, not the presence of process artefacts. It is not free, and it may not work. Agent refactorings are often wrong without verification or merely superficial [38], and the benefits of refactoring have been hard to attribute even for human teams [7, 52]. Whether it slows the decay is an empirical question, which the next section sets out to answer.

A checklist for Monday. For an architect about to start an agentic delivery loop, or already inside one:

1. Can a human use a deployed increment today? If not, make that the only priority.

2. Is "still runs and is still usable" checked on every merge, by a script that exercises a real user journey?
3. Is the backlog horizon two weeks or less, with work in progress capped?
4. Does each issue have five or fewer acceptance criteria?
5. Is there a fixed checkpoint, with a demo and a recorded decision, before each new wave?
6. Has every lesson from the last checkpoint become an executable check?
7. Can every document an agent must read be read in a few minutes?
8. Are gates proportional to the change, and do they test behaviour?
9. Do you know the cost per accepted outcome, not just per token?
10. Could you explain how the system works without asking an agent?

Implications for framework builders. Frameworks such as HVE Core already encode much of this: human stops between phases, plan critiques and context carried in files. Three additions would make the feedback-first path the easy one. A review phase could include using the running increment, not only reading the change. Artefact requirements could scale with the size and risk of a change, so that a two-line fix carries a two-line record. And lessons could have a first-class home as executable checks that every session loads, with a warning when the documents on an agent's read path exceed a budget.

8 Testing the harness: an experiment protocol

The proposed contribution is a controlled comparison of scheduled post-feature consolidation, not the invention of agent clean-up or objective-guided optimisation [78]. It tests whether an agent can perform a bounded part of reflective correction, rather than whether another agent can autonomously steer the whole development process. Lulla et al.'s registered-report protocol, which treats agent configurations as experimental arms, provides a model for specifying the comparison before observing results [62]. I designed the experiment with HVE Core's Experiment Designer as a Minimum Viable Experiment [49], following guidance for empirical studies that involve language models [6] and for software engineering experiments in general [94]. The execution choices below incorporate my decisions of 25 October 2026. I stopped execution and tagged the protocol. Registration is pending, and the exact tagged snapshot still needs to be recorded here. Earlier smoke, pilot and any other pre-registration runs will retain their original labels and timestamps. None will enter the registered confirmatory analysis, which will use only fresh runs started after registration. I will disclose what earlier data I saw and how it informed the design; a later registration does not retrospectively cover those runs.

The experiment kit is in a separate private repository. The ancillary protocol notes packaged with this paper are earlier proposals, not evidence of the executed or registered configuration. I will archive the reconciled protocol, suite, prompts, configuration and analysis snapshot with the registration once external deposit is authorised. The related work reviewed here does not isolate consolidation steps interleaved between successive feature additions. Darwin evaluates optimisation against measurable objectives [78]; SlopCodeBench shows that quality instructions alone shift where agents start but not how fast they degrade [72]. The harness tests a scheduled structural intervention and subsequent changeability instead.

8.1 Hypotheses

Table 7 lists the hypotheses in priority order. Each follows the Experiment Designer's form: what we believe, how we will test it, and the measurable outcome that would show it right or wrong.

Table 7: Hypotheses, compared after the final feature unless stated. A is feature-only delivery, B adds the consolidation harness, C adds a read-only review of equal budget. These decision rules must match the registered snapshot. Wrong means interval evidence rules out the minimum useful effect, not no significance; other evaluable outcomes are inconclusive.

ID	I propose	Right when	Wrong when
H1	The harness preserves working software	B's cumulative hidden pass rate is at least 10 points higher, 95% interval lower bound above zero	whole interval below 5 points
H6	Consolidated code absorbs an unplanned change better	Change-request success at least 20 points higher, or cost at least 25% lower at no lower success	Whole intervals rule out both 20-point gain and 25% saving
H2	Errors compound without consolidation	Open regressions trend upward in A; B has at least 50% fewer	Whole intervals rule out A's upward trend or 50% reduction
H3	Internal quality decays without consolidation	Positive slopes in A on three of four metrics; B's slope at most half of A's	Intervals rule out the required slope pattern
H5	The harness pays for itself	B's total cost at most 125% of A's and B cheaper per feature in the last third	Whole cost interval above 150%; late-saving interval at or below zero
H4	The harness leaves less to review	B has at least 20% fewer non-test lines; reviewers as accurate in 20% less time	Whole reduction intervals below 10% for size and time
H7	The consolidation package beats review-only	B beats C by at least half of the B–A gap	Intervals rule out the half-gap benefit

8.2 Design

Arms. Arm A delivers features one after another. Arm B runs the consolidation harness after every feature, with half the time budget of a feature step. B therefore has up to 50% more agent time per feature than A; the comparison is not time-matched. H5 compares their costs directly. I include arm C, which holds that extra time constant without refactoring and replaces consolidation with a read-only review of equal budget, to compare the complete consolidation package with review-only. B may add behaviour-pinning tests and writes an architectural `DESIGN.md`; C cannot change tests and writes issue-oriented `REVIEW.md`. Subsequent sessions read those different notes, so this comparison cannot isolate refactoring. Everything outside the post-feature intervention is identical: model and version, agent runtime, feature prompts, starting skeleton, tools and per-feature budgets.

The additional agent is a distinct session with a different responsibility, not a different pinned model or an independent source of truth. Its value must be demonstrated by behavioural and changeability outcomes, not inferred from the existence of another review role.

Runtime and budgets. I selected `claude-sonnet-5.5`, medium reasoning effort and Copilot CLI 1.0.103, with automatic updates disabled. The wall-clock limits are 30 minutes per feature, 15 per consolidation or review, and 45 for CR1. The pilot limit is 300 AI credits per session; any pilot-informed adjustment must be frozen and included in the registered configuration before the fresh study runs. The total pilot-and-first-pass ceiling is 2,500 AI credits, raised from an initial 1,000-credit proposal. I also stated a $500 monetary budget. Credit consumption and currency cost are not interchangeable without a verified billing conversion.

Suites. Suite S1 is a ten-feature retrofit case workbench drawn from the case domain, whose features deliberately interact: lifecycle, handoffs, returns, safety holds, resident consent, appointments with an escalation rule, role-based visibility, deterministic ranking and a hash-chained audit trail. The selected study uses all ten features and the full CR1, including the

ranking and export prerequisites in F9–F10. Each feature has visible example tests and hidden acceptance tests, including interactions with earlier features. Tests and reference solutions are intended to remain outside the agent's working repository. S2 is a library-lending suite reserved for a separately authorised main study; benchmark adaptation is deferred.

Isolation. I selected local execution rather than containers for the analysed runs. The CLI access rules and separation of working directories provide a weaker boundary than container isolation. They do not establish that hidden tests, references or outside resources are unreachable. Residual access risk limits validity and must accompany any results; an absence of leak flags is not proof of isolation.

Procedure. Every step runs in a fresh agent session, as in the case's coordinator, so that context travels only through the repository. A feature step may commit only when the visible tests pass. If it commits no green increment, the runner resets it, removes that feature's example tests and counts the feature as missing. Its hidden tests remain in the denominator. After every step, the external measurement process runs the cumulative hidden suite, which covers every feature scheduled so far, including missing features, and collects the metrics. A pilot calibrates budgets with one run per arm over ten features, without CR1. The selected first pass uses S1, arms A, B and C, with four independent runs per arm, ten features and CR1. This is the planned registered comparison, not data to pool with earlier started runs. An optional main study would use S1 and S2, five runs per arm and ten features, under a separate prospective plan and authorisation. With three runs per arm the exact two-sided Mann–Whitney floor is $2/C(6,3) = 0.10$, so it cannot reach $p < 0.10$; four runs reach 0.029 and five reach 0.008. These are attainability bounds, not a power analysis. The selected first pass has 120 feature, 40 consolidation, 40 review and twelve change-request sessions: 212 sessions and an upper bound of 89 agent-hours. The pilot adds fifty sessions and twenty agent-hours, making 262 sessions and 109 agent-hours combined. These bounds exclude measurement and scheduling overhead and do not establish affordability under the credit ceiling. Sample sizes and stopping rules must be registered before the fresh confirmatory runs; sample sizes will not be increased in response to observed effects.

End state and change request. H1's end state is after the last feature's post-feature step (a no-op in A), before the change request. Change-request success requires all its hidden tests to pass and no feature test that passed at that end state to fail afterwards.

Measures. Cumulative hidden pass rate, open regressions and how long they survive; mean cyclomatic complexity, the share of functions above complexity 10, duplicated lines and lint warnings per thousand lines; non-test lines, diff lines per step and documentation words generated; AI credits as the primary consumption measure, with tokens and elapsed time secondary; change-request success and cost; and, for S1, a blinded comprehension task in which reviewers answer five fixed questions about each final code base. Test integrity, through checksums of the example tests and counts of tests and assertions, is checked after every step. The planned code-metric settings are the runbook's isolated lint rules E, F, W, C90, B and SIM, duplication detection at fifty tokens and five lines, and complexity measured by lizard. Tool versions and settings must be fixed in the registration. I plan to invite two independent comprehension reviewers; their agreement and the assessment remain pending.

Intervention measures in prior work. Intervention-based robot learning provides useful measurement precedents. In HG-DAgger, the human decides when to take control, provides an uninterrupted corrective trajectory and explicitly returns control to the learned policy [51]. ThriftyDAgger distinguishes the number of interventions, human actions and robot actions per episode, alongside autonomous and intervention-aided success [47]. Its supervisor-burden model separates switching overhead from the length of assistance: $B(\pi) = C(\pi)[L + I(\pi)]$, where C is the expected number of switches from autonomous to supervisor control per episode, L is the round-trip switching latency and I is the expected number of supervisor actions per intervention. Latency is expressed in action timesteps, not coding-agent tokens. Fewer interventions can

therefore coexist with longer human involvement; continuous monitoring is another burden not captured by active correction time alone.

These studies motivate separating intervention incidence, frequency and effort, but are not evidence of coding-harness effectiveness. A frequency such as events per hour uses exposure time as its denominator; a human-control share uses human-controlled timesteps divided by all timesteps. Neither is the same as the proportion of tasks needing help. I adapt the latter to feature cycles and state the denominator explicitly, rather than claim one universal HIR formula. I retain failed cycles, unlike ThriftyDAgger's successful-episode summaries in its main evaluation; its supplement also reports totals over all trajectories.

Human intervention rate (HIR). As a secondary measure of harness quality, I record observed HIR: the proportion of attempted feature cycles receiving unplanned corrective human help. A cycle includes the feature and its assigned post-feature step, so B's additional sessions do not enlarge its denominator. Failed cycles remain included; repeated interventions in a cycle count once for the rate, with event counts recorded separately. For run r, let n_r be the number of attempted feature cycles and h_{rj} the number of qualifying corrective-help events in cycle j. Then

$$\mathrm{HIR}_r = \frac{N_{\mathrm{assisted},r}}{n_r} = \frac{1}{n_r} \sum_{j=1}^{n_r} \mathbf{1}\{h_{rj} > 0\}. \tag{1}$$

The indicator is one when the condition holds and zero otherwise; multiplying Equation 1 by 100 gives a percentage. By contrast, $\sum_j h_{rj}/n_r$ is events per cycle and can exceed one. I report event counts and active human minutes separately so that a brief clarification and a long repair do not appear equivalent merely because both affect one cycle. These are incidence and effort measures, not interchangeable estimates of total supervisory workload. If no cycle is attempted, the rate is undefined, not zero.

Primary runs receive no task-specific human coaching or code/test repair. I therefore also record an intervention-demand rate with the same denominator: cycles containing an explicit request for such help or a block requiring manual corrective action. With $d_{rj} = 1$ for such a cycle and zero otherwise,

$$\mathrm{IDR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} d_{rj}. \tag{2}$$

A request need not result in assistance, and an unsolicited human correction need not follow a recorded request; I code the two separately. Zero observed HIR under this policy is not evidence of autonomy. A timeout or automatic reset alone does not establish demand.

The intended sources are the session telemetry and retained logs. I plan to use Cowork to extract requests, blocks and actual interventions, then personally check classifications against source records and resolve ambiguous cases. The extraction must link each event to its cycle, phase, reason, action and active human minutes, using rules fixed before fresh confirmatory runs. Telemetry coverage of human-help requests and corrective actions, including actions outside the CLI, has not yet been verified. Missing records cannot be interpreted as zero interventions; incomplete coverage makes the affected measure not evaluable. Routine launches, planned evaluation, required approvals and safety stops are recorded separately, not treated as corrective rescue. Any corrective help supplied is a protocol deviation; I retain the run, label it assisted and report its outcomes separately from unassisted estimates. Such deviations do not trigger replacement runs and may leave the confirmatory comparison not evaluable at its predeclared sample size. I report change-request interventions separately, alongside completion, missing features, early termination and unattempted cycles. Lower HIR alone is not better: it must not reward silent failure or discourage necessary human judgement.

8.3 Analysis and bias controls

The difference in final cumulative pass rate is estimated with bootstrap intervals that resample runs within arm. S1 trends are estimated with mixed-effects models that include an arm-by-feature-index interaction and run as a random intercept. A later multi-suite study would treat suite as a fixed effect and resample within suite and arm. Because code generation is not deterministic, even at temperature zero [75], every condition is run several times and intervals are reported rather than single values. The S1 first pass has a fixed sample size; pilot and all pre-registration runs are excluded from its confirmatory analysis. A later main study would also exclude those earlier data. Overlapping intervals do not trigger extra confirmatory runs: any extension is separately labelled exploratory and is not pooled into the original analysis. For H1, validation requires a gain of at least 10 percentage points and a 95% interval wholly above zero; invalidation requires its upper bound to be below 5 points. All other evaluable H1 outcomes are inconclusive and route to adjust, not no-go. For every hypothesis, validation requires the estimate to meet its stated threshold and the 95% interval for the benefit to exclude zero in the beneficial direction. Invalidation requires the whole interval to fall short of the stated minimum effect of interest, applying the joint or alternative conditions in Table 7; absence of significance is not invalidation. For H5 I assess headroom below the 150% cost ceiling and late-feature savings, while retaining the 125% validation threshold. An interval spanning zero, without ruling out a useful effect, is inconclusive. The planned controls are randomised interleaving of arms on the same machine; frozen hidden tests, metrics and analysis before fresh registered runs; exclusion of pilot data; blinded comprehension assessment; reporting of every run; and a divergence log. Independent suite review, verified access boundaries and the final analysis checks are still pending, not completed safeguards. Each hypothesis is recorded as validated, invalidated, mixed, inconclusive or not evaluable, and the decision as go, no-go or adjust.

HIR is descriptive, with no new hypothesis threshold or adoption gate. I report observed and demand rates per run, human minutes and the B–A demand-rate difference with a 95% interval, resampling whole runs within suite and arm. The rates accompany correctness and cost, rather than replacing them.

The protocol uses one proprietary model and no open-weights baseline. This limits reproducibility and transfer to other models; pinning the exposed version does not freeze a provider's service [6].

8.4 Status

This version presents the protocol, not study results. As of 25 October 2026 I had stopped execution and reported tagging the protocol. Registration was still pending. Earlier real-agent runs are retained as pre-registration evidence under their original labels and timestamps, not retrospectively declared preregistered. I will start fresh runs only after registration and exclude every earlier run from confirmatory analysis. The exact tag and registration record remain to be supplied. Results, including null and negative results, will be added in a revision after checked analysis; a conformant run that invalidates a hypothesis remains a valid outcome.

9 Discussion: the existential crisis, revisited

9.1 What worked

It would be wrong to read this case as a failure of agents. The research phase assembled, in days, a corpus and analysis that would have taken me weeks. The role agents' challenge killed a bad idea, five end-to-end agents, before any code was written. The agents refused to invent approvals, figures or company facts, and recorded the limits of their evidence with a care many

human teams lack. The MVP was built and reshaped in a day. Each of these successes had the same shape: a short loop with a human in it.

9.2 What remained mine in this case

Looking back over six weeks, I still supplied several kinds of reflective work. I decided what not to build. I noticed that something was wrong, not from the reports, which were accurate and full of caveats, but from the plain fact that I could not use the system. I held the theory of the system, in Naur's sense: which lessons mattered and why the sign-in had worked on 12 October. And I was the person responsible for designing the feedback loop and deciding what useful feedback would look like in this case.

That is close to Beck's distinction between a discipline's visible output and the less visible work of understanding, simplifying and building abstractions [15], and to Martin's insistence that programmers still write the tests first and know their customers [65]. The agents were excellent at the visible part. They produced proofs of work: pull requests, artefacts, receipts and honest caveats. In this case those records did not replace the understanding I gained from using a running system. That does not establish a general limit on what agents can do. It identifies responsibilities that could be candidates for bounded delegation, not proof that they are intrinsically or permanently human.

9.3 Can another agent restore futures?

Beck's alternation suggests a different purpose for an additional agent: preserving the options that feature execution consumes, rather than accelerating the next backlog item [14, 16]. Such an agent would need to inspect the accumulated system, detect drift against intended behaviour and take or recommend a corrective action. An agent that only reads generated records and approves them could reproduce the proxy capture observed here. I propose that a steering role needs behavioural evidence, bounded authority to act and checks that can disconfirm its judgement. More agents do not by themselves supply those conditions.

Darwin already delegates parts of this steering: agents turn observations into hypotheses and guide which candidate to explore next. Its boundary is important. Search improves an explicit objective within an existing architecture; engineers can steer objective setting and evaluation packaging, World Model updates are reviewed, and production output still enters human review queues [78]. This narrows the open question from whether agents can perform any reflection to whether their correction preserves useful future options as requirements change. Performance gains do not establish that result, but they are a relevant precedent for bounded delegation.

The present experiment takes one step towards that question. It asks whether an agent-led consolidation package leaves software more able to absorb an unforeseen change. H6 is the closest probe of preserved future options; complexity and duplication are supporting proxies, not a direct measure of all possible futures. The fixed post-feature cadence does not test whether the agent can decide when to interrupt delivery, challenge a requirement, change the backlog or restore a usable deployment. A positive result would support this bounded consolidation role, not unrestricted process steering. A null or negative result would challenge this particular intervention, not prove that reflective work cannot be delegated to agents.

9.4 If drift is contained, what remains for us?

Even if an agent can contain code drift, the human questions do not disappear. Better software outcomes would not, by themselves, establish how people should participate in producing them. Three questions remain open.

What is our role in the process? If agents can both implement features and consolidate the result, do we primarily choose problems, frame intent and judge consequences, or does meaningful

participation also require working on the system itself? One possible direction is to spend more of the human effort on understanding users, challenging goals, designing experiments and deciding what not to build. That is a proposal for using the capacity released by automation, not evidence that these activities belong exclusively to humans or that implementation no longer contributes to understanding.

How much should we intervene? Too little involvement may leave us unable to recognise a wrong direction; intervening in every step may prevent the delegation we hoped to achieve. A possible starting point is a cadence of hands-on product use, with additional intervention when uncertainty, consequences or difficulty of reversal increase. The appropriate balance remains untested. HIR and intervention demand describe help supplied or requested; they do not identify an optimal level of involvement. A timely intervention that challenges an assumption or protects a user may be valuable, even if it raises the measured rate.

What will we learn, and how? We may learn more about the domain and the consequences of our choices while learning less through implementation. But if agents also perform the correction, where will the understanding needed to question them come from? Naur's theory building and Bainbridge's warning about eroded operator skills make this a practical concern as well as a philosophical one [5, 70]. Possible practices include predicting behaviour before trying an increment, investigating surprises and periodically explaining or changing the core design without agent assistance. Whether these preserve understanding, especially for people still learning the craft, needs separate study; producing another design note would not establish that they do.

These questions concern agency, expertise, responsibility and the meaning of the work, not only throughput. Some aspects could be studied through human task performance, comprehension and learning over time; what participation we value also requires judgement that a code-quality result cannot settle. This paper offers directions for that inquiry, not answers. Neither the case nor the proposed experiment determines what humans should do, how much we should intervene or what we will learn if code drift is contained.

9.5 The crisis reframed

The existential question is usually put as whether machines will replace developers. I ask instead which parts of reflection and course correction agents can perform reliably, and how to tell when they are helping rather than reinforcing the original mistake. In this case, producing more work did not substitute for feedback. Designing, protecting and interpreting that feedback remained my responsibility, but that observation is a starting point for testing delegation, not a boundary I assume agents cannot cross. The old practices, testing, small steps, tidying and walking skeletons, offer ways to make such delegation observable and falsifiable. Beck's 2023 recalibration was about which ten per cent of his skills gained leverage [11]; in this case, the skills that mattered were the ones that keep a feedback loop short. Lamanna's advice to judge AI by the same business outcomes as before [54], and Lanzetta's description of the engineer as the architect of intent who builds the system in which agents work [56], point in the same direction.

10 Threats to validity

Construct validity. Word counts and a hedging lexicon are crude proxies for documentation load and for the content of status records, and a proportion of hedged statements is appropriate caution. I report them as indicators, alongside qualitative evidence, not as measures of quality.

Internal validity. I designed the coordinator and its gates, so the outcome reflects my configuration as much as the framework or the models. HVE Core's defaults (a manual mode, human stops and git-ignored tracking files) point the other way; I overrode them. The two periods differ in length and in the work attempted, model and tool versions changed during the study, and other work on the capstone, such as presentations, ran in parallel.

External validity. This is a single case: one developer, a fictional but regulation-heavy domain, and a capstone whose rubric rewards documentation as well as software. Teams with dedicated product owners, users or operations staff may experience the mechanisms differently. The field observations in Section 6 suggest the mechanisms are recognised elsewhere, but they are not evidence of prevalence.

Reliability and researcher bias. The repository, issues and log are public, and Appendix A defines the measures so that they can be recomputed. The experiment's own threats, and their mitigations, are part of its protocol (Section 8). As participant and author I may over-weight episodes that support a tidy story; the log's contemporaneous, append-only form limits but does not remove that risk. AI-assisted summaries and interpretation may also select or smooth evidence towards a coherent story. My selective review is not an independent analysis, and mechanism naming is not systematic qualitative coding. AI assistance in analysis and drafting is disclosed below; source checking does not establish an unbiased interpretation.

11 Conclusion

An autonomous, spec-driven agent loop can produce an enormous amount of software engineering activity, requirements, plans, reviews, receipts and merged pull requests, without producing software that its owner can use. In this case the evidence points towards a configuration that measured progress by artefacts, answered failures with more gates, kept lessons in prose, and had no invariant requiring a usable system. It does not rule out the rival explanations or establish that model capability was irrelevant. These patterns fit old lessons of software engineering; I propose that agentic throughput can amplify them. My practical response is to make feedback the primary output of every cycle: a running increment, a human checkpoint and executable lessons. Objective-guided optimisation provides a bounded precedent for agent-led correction [78]; the further question is whether such correction preserves changeability as features accumulate. Accountability for goals and consequential decisions remains distinct from delegating bounded corrective work; the case does not establish that only humans can perform that work.

The experiment operationalises a bounded version of that question: can an agent that consolidates after every feature keep agent-built software working, understandable and changeable at an acceptable cost? It tests whether an additional agent can help restore future options, not whether it can choose the right product or autonomously recover every delivery failure. Section 8 sets out how to find out. The larger question of agent-led steering remains open; the first requirement is evidence that the additional agent improves outcomes rather than adds another layer of work. Even then, what people should do, when they should intervene and how they will continue to learn remain open questions. The proposals in Section 9.4 are starting points, not conclusions of this study.

Acknowledgements

I thank the colleagues and partners whose questions shaped this report.

Use of generative AI

I used GitHub Copilot with HVE Core agents to build the system under study. The coordinator's recorded RPI worker selector from 21 October is `claude-sonnet-5.5`; its own session used Automatic selection, not a fixed model name. The deployed assistant exercise on 14 October used `gpt-4.1-mini-2025-04-14`, distinct from the coding agent. I used Microsoft 365 Copilot for summarisation and finding relevant reference documents, and Copilot Cowork for slide generation. Log analysis, development of the mechanisms and manuscript drafting took place with GitHub

Copilot in VS Code, using the reported model labels in Section 3. I supplied the observations and explanations; Copilot named and organised the patterns. This was not a formal qualitative coding exercise. I read Beck's mathematicians essay, watched the related full talk and confirmed the paraphrase here. I checked the case's repository figures and Azure spend; I have not yet independently checked the cited research statistics or the log-growth figure.

GitHub Copilot revised draft v3 and this author clarification, checked new reference metadata against source pages, and recomputed protocol arithmetic with the supplied scripts. Bibliographic checks do not establish the validity of every claim in a source. For future HIR analysis I plan to use Cowork to extract events and will personally check classifications against source records and resolve ambiguous cases. These are planned checks, not completed results. Copilot also compared Pavlenko et al.'s version 1 paper against the manuscript and revised the related-work, learning and bounded-steering discussion. Checking that source's reported scope and metadata is not an independent verification of its experiments. I selected the case, made the analytical judgements and take responsibility for the content.

Data availability

The repository analysed is public at https://github.com/jmservera/ama; this report uses commit `29c6b512f5e68733bb2cc1882343db39f5eb2cad`. The log-measurement and figure-generation scripts are provided with the source. The ancillary experiment notes in HVE Core's Minimum Viable Experiment format are earlier proposals, not the tagged study record. The separate study kit is held in a private repository; it is not yet a public replication archive. The registered snapshot, sanitised data and exact run provenance will need an authorised deposit before a reproducibility claim can be made.

References

[1] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux. Spec-driven development as prescribed and as practiced. arXiv:2608.25202, 2026.

[2] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas. Manifesto for agile software development, 2001. https://agilemanifesto.org/.

[3] J. Young. Effective harnesses for long-running agents. Anthropic, 26 November 2025. https://www.anthropic.com/research/effective-harnesses-for-long-running-agents.

[4] A. Bacchelli and C. Bird. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 712–721. IEEE, 2013. doi:10.1109/ICSE.2013.6606617.

[5] L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi:10.1016/0005-1098(83)90046-8.

[6] S. Baltes et al. Guidelines for empirical studies in software engineering involving large language models. arXiv:2508.15503, 2025. Accepted in *Empirical Software Engineering*.

[7] G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba. An experimental investigation on the innate relationship between quality and refactoring. *Journal of Systems and Software*, 107:1–14, 2015. doi:10.1016/j.jss.2015.05.024.

[8] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[9] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2nd edition, 2004.

[10] K. Beck. `test && commit || revert`. Medium, 28 September 2018. https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864.

[11] K. Beck. 90% of my skills are now worth $0. *Software Design: Tidy First?* (newsletter), 19 April 2023. https://tidyfirst.substack.com/p/90-of-my-skills-are-now-worth-0.

[12] K. Beck. *Tidy First? A Personal Exercise in Empirical Software Design.* O'Reilly Media, 2023.

[13] K. Beck. Augmented coding: Beyond the vibes. *Software Design: Tidy First?* (newsletter), 25 June 2025. https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes.

[14] K. Beck. Why does development slow? *Software Design: Tidy First?*, 19 November 2025. Author's summary at https://kentbeck.com/summaries/why-does-development-slow/.

[15] K. Beck. Mathematicians, here's a way to think about your existential crisis. *Software Design: Tidy First?* (newsletter), 29 September 2026. https://newsletter.kentbeck.com/p/mathematicians-heres-a-way-to-think.

[16] K. Beck. Software engineering in the age of AI. Talk at Prodacity 2026, Nashville, TN, 25–27 August 2026. Video: https://www.youtube.com/watch?v=F8fBgDCf2Y4; transcript: https://www.rise8.us/videos/kent-beck-software-engineering-in-the-age-of-ai.

[17] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089, 2025.

[18] T. Besker, A. Martini, and J. Bosch. Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 105–114. ACM, 2018. doi:10.1145/3194164.3194178.

[19] B. Böckeler. Understanding spec-driven-development: Kiro, spec-kit, and Tessl. martinfowler.com, 15 October 2025. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html.

[20] M. Borg, D. Hewett, N. Hagatulah, N. Couderc, E. Söderberg, D. Graham, U. Kini, and D. Farley. Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31(6):161, 2026. doi:10.1007/s10664-026-10889-1.

[21] F. P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi:10.1109/MC.1987.1663532.

[22] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.

[23] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[24] A. Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams.* Addison-Wesley, 2004.

[25] J. Cohen. *Best Kept Secrets of Peer Code Review.* Smart Bear Inc., 2006.

[26] W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92 (Experience Report)*, pages 29–30. ACM, 1992. doi:10.1145/157709.157715.

[27] D. Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2 September 2025. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.

[28] J. Ding, S. Long, C. Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. arXiv:2512.12730, 2025.

[29] DORA. Accelerate state of DevOps report 2024. Google Cloud, 2024. https://dora.dev/research/2024/dora-report/.

[30] DORA. State of AI-assisted software development 2025. Google Cloud, 2025. https://dora.dev/dora-report-2025.

[31] Google Cloud. Announcing the 2025 DORA report. Google Cloud Blog, 23 September 2025. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report.

[32] S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron, and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi:10.1109/32.895984.

[33] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri. LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 50(9):2254–2268, 2024. doi:10.1109/TSE.2024.3428972.

[34] M. Fowler. Design stamina hypothesis. martinfowler.com, 20 June 2007. https://martinfowler.com/bliki/DesignStaminaHypothesis.html.

[35] M. Fowler. Some thoughts on LLMs and software development. martinfowler.com, 28 August 2025. https://martinfowler.com/articles/202508-ai-thoughts.html.

[36] J. Gall. *General Systemantics: An Essay on How Systems Work, and Especially How They Fail.* General Systemantics Press, Ann Arbor, MI, 1975.

[37] E. Gamma and K. Maetzel, interviewed by W. Bos and S. Tolinski. Fork yeah! Microsoft open sourcing Copilot. *Syntax* podcast, episode 903, 19 May 2025. https://syntax.fm/show/903/fork-yeah-microsoft-open-sourcing-copilot.

[38] D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.07832.

[39] GitClear. AI copilot code quality: 2025 data suggests 4x growth in code clones. Research report, GitClear, 2025.

[40] E. M. Goldratt and J. Cox. *The Goal: A Process of Ongoing Improvement.* North River Press, 1984.

[41] C. A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*. Reserve Bank of Australia, Sydney, 1975.

[42] N. Harvey and D. DeBellis. Announcing the 2024 DORA report. Google Cloud Blog, 22 October 2024. https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report.

[43] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the International Conference on Mining Software Repositories (MSR '26)*. ACM, 2026. doi:10.1145/3793302.3793349. arXiv:2511.04427.

[44] L. G. Heander, M. Borg, E. Söderberg, and C. Rydenfält. CodeHealth predicts the token cost of agentic code review: A pre-registered study of 465 agent-authored pull requests. arXiv:2610.21914, 2026.

[45] I. Herraiz, D. Rodriguez, G. Robles, and J. M. Gonzalez-Barahona. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Computing Surveys*, 46(2):28, 2013. doi:10.1145/2543581.2543595.

[46] K. Hong, A. Troynikov, and J. Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, July 2025. https://trychroma.com/research/context-rot.

[47] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 598–608. PMLR, 2022. https://proceedings.mlr.press/v164/hoque22a.html.

[48] A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.

[49] Microsoft. HVE Core: HyperVelocity Engineering for GitHub Copilot. GitHub repository and documentation, accessed 24 October 2026. https://github.com/microsoft/hve-core; https://microsoft.github.io/hve-core/.

[50] A. Karpathy. Post on X introducing "vibe coding", 2 February 2025. https://x.com/karpathy/status/1886192184808149383.

[51] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. arXiv:1810.02890v2, 2019 (first posted 2018).

[52] M. Kim, T. Zimmermann, and N. Nagappan. An empirical study of refactoring challenges and benefits at Microsoft. *IEEE Transactions on Software Engineering*, 40(7), 2014. doi:10.1109/TSE.2014.2318734.

[53] P. Laban, H. Hayashi, Y. Zhou, and J. Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.06120.

[54] C. Lamanna. Changing how we build. LinkedIn article, 5 August 2026. https://www.linkedin.com/pulse/changing-how-we-build-charles-lamanna-xpitc/.

[55] M. Lanzetta. Growing engineers in the hypervelocity era. LinkedIn article, 21 April 2025. https://www.linkedin.com/pulse/growing-engineers-hypervelocity-era-mike-lanzetta-qux0c/.

[56] M. Lanzetta. The hypervelocity orchestra. LinkedIn article, 25 February 2026. https://www.linkedin.com/pulse/hypervelocity-orchestra-mike-lanzetta-p0kvc/.

[57] H.-P. Lee, A. Sarkar, L. Tankelevitch, I. Drosos, S. Rintel, R. Banks, and N. Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–22. ACM, 2025. doi:10.1145/3706598.3713778.

[58] M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi:10.1109/PROC.1980.11805.

[59] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski. Metrics and laws of software evolution: The nineties view. In *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997. doi:10.1109/METRIC.1997.637156.

[60] H. Li, H. Zhang, and A. E. Hassan. AIDev: Studying AI coding agents on GitHub. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026. arXiv:2602.09185. doi:10.1145/3793302.3797249.

[61] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:10.1162/tacl_a_00638.

[62] J. L. Lulla, M. Galster, J. M. Zhang, S. Baltes, and C. Treude. The impact of configuring agentic AI coding tools on build-vs-buy decisions: A study protocol. ESEM 2026, Registered Reports track. arXiv:2606.03907, 2026.

[63] R. C. Martin. *We, Programmers: A Chronicle of Coders from Ada to AI*. Addison-Wesley Professional, 2024.

[64] R. C. Martin. AI out-codes you; deal with it. Video post on X (@unclebobmartin), 20 April 2026. https://x.com/unclebobmartin/status/2046206145597972849.

[65] R. C. Martin. Why hardcore technical focus still matters big time. *Product Engineers* podcast, 29 September 2026. https://productengineers.com/episodes/season2/why-hardcore-technical-focus-still-matters-big-time-uncle-bob/.

[66] J. Becker, N. Rush, T. Cunningham, D. Rein, and K. Mahamud. We are changing our developer productivity experiment design. METR blog, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/.

[67] A. Murphy. The five stages of losing our craft. Blog post, 8 February 2026. https://andrewmurphy.io/blog/the-five-stages-of-losing-our-craft.

[68] A. Murphy. If you thought the speed of writing code was your problem – you have bigger problems. Blog post, 17 March 2026. https://andrewmurphy.io/blog/if-you-thought-the-speed-of-writing-code-was-your-problem-you-have-bigger-problems.

[69] E. Murphy-Hill, J. Butler, and A. Savelieva. Adoption and impact of command-line AI coding agents: A study of Microsoft's early 2026 rollout of Claude Code and GitHub Copilot CLI. arXiv:2607.01418, 2026.

[70] P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985. doi:10.1016/0165-6074(85)90032-8.

[71] R. Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, 11 February 2026. https://openai.com/index/harness-engineering/.

[72] G. Orlanski, D. Roy, A. Yun, C. Shin, A. Gu, A. Ge, D. Adila, N. Roberts, F. Sala, and A. Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755, 2026.

[73] K. Beck, interviewed by G. Orosz. TDD, AI agents and coding with Kent Beck. *The Pragmatic Engineer* podcast, 11 June 2025. https://www.youtube.com/watch?v=aSXaxOdVtAQ.

[74] A. Osmani. The 70% problem: Hard truths about AI-assisted coding. *Elevate* (newsletter), December 2024. https://addyo.substack.com/p/the-70-problem-hard-truths-about.

[75] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):42, 2025. doi:10.1145/3697010.

[76] R. Pan, J. Zhang, L. Wu, Y. Zhang, H. Zhang, and L. Zhang. Do modifications made by coding agents make subsequent software maintenance harder? arXiv:2610.14768, 2026.

[77] D. L. Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287. IEEE, 1994. doi:10.1109/ICSE.1994.296790.

[78] A. Pavlenko, B. Crivat, B. Haynes, C. Curino, F. Psallidas, J. Slawinski, J. Freischuetz, L. Pereira Sanchez, M. Weimer, M. Demarne, M. Jasny, M. Gandhi, M. Bovykin, M. Milletari, P. Ghosh, Q. Bai, R. Ramakrishnan, R. Pandita, S. Matusevich, S. Venkataraman, S. Krishnan, M. T. Mahmood, T. Bang, V. Emani, X. Zhao, and Y. Zhu. Towards an AI software factory for data systems. arXiv:2609.36323v1, 28 September 2026. https://arxiv.org/abs/2609.36323v1.

[79] S. R. O. Peralta, F. Hoshi, H. Washizaki, N. Ubayashi, I. Kondo, Y. Higo, H. Mukai, N. Yoshida, K. Kusama, H. Tanaka, and Y. Fan. Why are agentic pull requests merged or rejected? An empirical study. arXiv:2605.22534, 2026.

[80] W. W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, August 1970.

[81] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.

[82] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli. Modern code review: A case study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 181–190. ACM, 2018. doi:10.1145/3183519.3183525.

[83] A. Sarkar. AI should challenge, not obey. *Communications of the ACM*, 67(10):18–21, 2024. doi:10.1145/3649404.

[84] J. M. Servera Bondroit. AMA Case Study 48: research and solution design for the Zava Forward retrofit programme. GitHub repository, commit 29c6b51, 24 October 2026. https://github.com/jmservera/ama.

[85] P. Ralph et al. Empirical standards for software engineering research. arXiv:2010.03525v2, 2021 (first posted 2020). ACM SIGSOFT Empirical Standards: https://www2.sigsoft.org/EmpiricalStandards/.

[86] A. Simkute, L. Tankelevitch, V. Kewenig, A. E. Scott, A. Sellen, and S. Rintel. Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interaction. *International Journal of Human–Computer Interaction*, 41(5):2898–2919, 2025. doi:10.1080/10447318.2024.2405782.

[87] A. Sinha, A. Arun, S. Goel, S. Staab, and J. Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.09677.

[88] M. Strathern. 'Improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[89] L. Tankelevitch, V. Kewenig, A. Simkute, A. E. Scott, A. Sarkar, A. Sellen, and S. Rintel. The metacognitive demands and opportunities of generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–24. ACM, 2024. doi:10.1145/3613904.3642902.

[90] Thoughtworks. Complacency with AI-generated code. Technology Radar (Techniques, Hold), 2024–2025. https://www.thoughtworks.com/radar/techniques/complacency-with-ai-generated-code.

[91] A. Tornhill and M. Borg. Code red: The business impact of code quality. A quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt (TechDebt '22)*, pages 11–20. ACM, 2022. doi:10.1145/3524843.3528091.

[92] M. Waseem, A. Ahmad, K.-K. Kemell, J. Rasku, S. Lahti, K. Mäkelä, and P. Abrahamsson. Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use. arXiv:2512.11922, 2025.

[93] S. Willison. Vibe engineering. Simon Willison's Weblog, 7 October 2025. https://simonwillison.net/2025/Oct/7/vibe-engineering/.

[94] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi:10.1007/978-3-642-29044-2.

[95] W. Yan. Don't build multi-agents. Cognition blog, 12 June 2025. https://cognition.ai/blog/dont-build-multi-agents.

[96] H. Yu, L. Liu, X. Jiang, Y. Jia, S. Wang, P. Qian, and Y. Chen. Habituation at the gate: Rising approval and declining scrutiny in human review of AI agent code. KDD 2026 Workshop on Agentic Software Engineering, 2026. arXiv:2606.22721.

[97] Z. Zhong, A. Raghunathan, and N. Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.20270.

A Measures and lexicon

Log entries. The knowledge-base log (`docs/reference/log.md`) was split into entries at each second-level heading that begins with a date. An entry belongs to the date in its heading. Word counts use whitespace tokenisation of the entry, heading included.

Hedging phrases. Each entry was searched, case-insensitively and on word boundaries, for these patterns: *unmet*; *not accepted*; *not inferred*; *is inferred*; *remain(s) open*; *remain(s) unproved*; *remain(s) unverified*; *not claimed*; *not established*; and *no* followed, within forty letters, spaces, commas, slashes or hyphens, by *is* or *was* and then *claimed* or *inferred*. Density is matches per 1,000 words.

Reference tracing. An entry cites the simplicity requirements if it contains an identifier from SA-001 to SA-010 or the word *simplicity*. It cites the development profile if it contains *deploymentProfile*, *development profile* or *allowPublicNetworkAccess*.

Repository counts. Issue and pull-request counts come from the GitHub search API on 24 October 2026 (for example, `is:pr is:merged merged:>=2026-10-15`). Document sizes are whitespace word counts of the files at commit `29c6b51`.

Proposed measures. *Time to first human use*: elapsed time from the start of a delivery cycle to the first time a person uses the running increment for its purpose. *Age of the last usable increment*: time since a person last verified that the deployed system could be used for its main journey. *Behaviour-changing merge ratio*: the share of merged pull requests that change what a user can observe in the running increment. *Cost per accepted outcome*: total model, tool and cloud cost divided by the number of outcomes a human accepted in the same period.

B Consolidation step prompt

The prompt used for the consolidation step of arm B (Section 7.1). Braces mark parameters set by the experiment runner.

```
Do not add or change features. Your job is to leave the code better than
you found it without changing its behaviour.

1. Run ‘make check‘, then read reports/quality.md (functions above
   complexity 10, duplicated blocks, lint warnings, file sizes).
2. Review the last feature’s diff (‘git diff feature-start feature-end‘,
   tags the runner sets around each feature step) and the hotspots
   against this checklist: duplicated logic; long or complex functions;
   unclear names; dead code; responsibilities in the wrong place, or
   rules from one feature leaking into another; tests that check
   implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through
   ‘./tcr "<message>"‘: it runs all tests and commits if they pass, or
   reverts if they fail.
4. Never modify or delete files in tests/examples/. You may add tests that
   pin current behaviour before you refactor.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where
   each business rule lives, and known debts. Replace; do not append.
6. Stop when the quality budget is met (no function above complexity 10,
   duplicated lines below 3%, zero warnings in the configured lint rules)
   or when your time budget ({consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what
   debt remains.
```

Table 8: Rival explanations. Evidence against is a limit on an explanation, not a reason to rule it out.

Explanation	Evidence for	Evidence against	How this report treats it
Migration decision	ADR 0006 (issue #196) froze feature work in the working MVP and prioritised a new layout.	The new layout passed health and restart checks; this did not establish usability.	A direct rival to autonomy alone; migration and coordination changed together.
Capstone rubric	The rubric rewards research and documentation; the scorer gave completeness full marks without usable sign-in.	The brief also asks for a working demonstrator.	A possible incentive for artefact production, not a measured incentive effect.
My coordinator and gate rules	I required committed RPI records and a full gate; PR #445 needed an exception for a small fix.	The MVP used agents successfully; HVE Core defaults retain human stops.	Limits attribution to agents or HVE Core; this is my configuration.
Model and tool changes	Versions changed during the study.	There is no matched rerun under fixed versions.	An unresolved confound; configuration files cannot establish every session's actual model.
CI and cloud budgets	Validation moved locally in issue #413; cloud costs prompted a higher ceiling and reduced resources.	Merges continued; local checks did not establish deployed usability.	A plausible constraint on delivery, not evidence that more budget would resolve it.
Identity and sign-in setup	The new deployment lacked registration and staff bindings; issue #349 distinguished a redirect from sign-in.	Those steps had worked in the MVP on 12 October.	A concrete deployment obstacle and possible lost learning, not proof of general code decay.